

# 从C到C++

## 1.cout和cin

C语言中的printf和scanf写法很优秀，但是操作很令人头疼。直接用c++中的cout和cin则好很多。

```
1  #include <iostream>
2  using namespace std;
3  /*
4   * namespace是名空间的意思，std是该名空间的名字
5   * 我们在课堂上讲了名空间的意义，以及好处。
6   * 简单来说，名空间就是一个空间区域或者容器
7   * 它将一些函数、类型全部放在某个区域，
8   * 让这些东西都有了归属。
9   */
10 int main(){
11     int a = 10;
12     float b = 10.0;
13     cout << a <<endl; // endl = endlene, 也就是换行
14     cin >> a; // 输入，不需要取地址。
15     return 0;
16 }
```

如果没有使用名空间，那么写法如下：

```
1  #include <iostream>
2  int main(){
3      int a = 10;
4      float b = 10.0;
5      std::cout << a <<std::endl;
6      std::cin >> a;
7      return 0;
8  }
```

## 2.字符串string

前面也说过，一门语言的受市场的欢迎程度，很大一部分取决于它是否给用户提供了方便的字符串处理和结构化数据处理机制（即函数）。

```
1  #include <iostream>
2  #include<string> // 注意，这里不是string.h
3  using namespace std;
```

```

4
5 int main(){
6     string name = "wangzhendong";
7     string job = " is a teacher";
8     string all = name+job; //+号在这里是连接符的意思，不是运算符
9     cout<<all<<endl;
10
11     //请注意，string在这里是一个类，这里再也没有strlen了，而应该是 size
    或者length
12     cout<<"size:"<<all.size()<<endl;    //size:25
13     cout<<"length:"<<all.length()<<endl; //length:25
14
15
16     //判断字符串是否为空
17     cout<<all.empty()<<endl; //0
18
19     //像数组一样使用[]
20     int i = 0;
21     for(;i<all.size();i++){
22         cout<<all[i];
23     }
24
25     //[]是没有边界检查的，但是at函数有
26     cout<<all.at(all.size()); // 提示 out of range。即数组越界。
27
28
29     //获得子字符串
30     cout<<all.substr(4,9); // zhendong
31
32     // 查找
33     cout<<(int)all.find("wzd"); // 没找到返回-1
34     cout<<all.find("wang"); // 找到了返回开始匹配的index
35     return 0;
36 }

```

C++ 没有提供字符串切割 split函数，这个函数很重要，我们在后面实现。

### 3. C with class

c++的本名是 C with class，其做法非常简单粗暴。就是将 struct 改成了 class。不过，这不是一个形式主义，而是将函数写进入了 class 之中。

```

1 struct Person{
2     int id;
3     char *name;
4 };

```

转换后：

```

1  #include<iostream>
2  using namespace std;
3  class Person{
4
5      /*
6       * public 是公众的，与之对应的是protected（受保护的）和private（私有
       的）
7       * 这三个关键字是控制 class 中的 id name getId函数和setId函数的访问
       权限
8       * public 指的是到处可以访问
9       * private 指的是只有在class 内部可以访问。
10      * 较为折中的是，protected，它在内部访问，且继承Person的子类可以使用
11      * 现阶段，我们统一定义成public
12      */
13      public:
14          int id;
15          char* name;
16          /*
17           * this是一个指向自己的指针，往后在学习面向对象时，都会有this指向
           自己
18           * 或者不叫this，叫self
19           */
20          int getId(){
21              return this->id;
22          }
23
24          void setId(int id){
25              this->id = id;
26          }
27 };
28
29 //使用
30 int main(){
31     Person p1;
32     p1.setId(10);
33     cout<<p1.getId()<<endl; // 10
34
35     //也可以定义指针
36     /*当定义指针后，不能用 . 而要用 -> */
37     Person *p = &p1;
38     p->getId(); //
39     return 0;
40 }

```

实际上C++项目的代码不是这样写的。一般会分多个文件。

项目，我们在课堂上已经生成了，在这里就不重复了——也没办法写。

仍以Person为例，看看它的项目构造是怎样的。

## Person.h文件

```
1  #ifndef PERSON_CPP
2  #define PERSON_CPP 1
3  class Person{
4      public:
5          int id;
6          char* name;
7          int getId();
8          void setId(int id);
9  };
10 #endif
11
12 /*
13  *1.这个文件中只对Person进行定义，包括函数的声明
14  *2.这里面有个宏定义。作用是让Person类的定义唯一。这一点我们在课堂上也讲
15  */
```

## Person.cpp文件

```
1  #include"Person.h"
2  int Person:: getId(){
3      return this->id;
4  }
5
6  void Person:: setId(int id){
7      this->id = id;
8  }
```

## main.cpp文件

```
1  #include<iostream>
2  using namespace std;
3  int main(){
4      //TODO
5      return 0;
6  }
```

## 4. new 和delete

new相当于malloc，delete相当于free，都是操作堆（LEAP）上的内存。这一点，咱们在之前的课程已经讲了很明白了。

但是从哲学角度出发，new 和malloc是完全两回事，中间过程较为复杂，我就说了。

```

1  int main(){
2      /*1.new一个对象*/
3      Person *p = new Person();
4      delete p;
5
6      /**2.new一个数组*/
7      int *a = new int[10];
8      delete a;
9      return 0;
10 }

```

## 5.STL

STL就是标准模板库的缩写——Standard Template Library

**Standard (标准) :**

C/C++语言中总是习惯将它自带的东西定义为“标准”，如 `stdio = standard input and output`,

再比如, `using namespace std` 相当于 `using namespace standard`.

总之, 在C/C++语言中, 如果遇到了 `std`, 就意味着 `standard`。

**Template (模板) :**

什么是模版? 其实咱们课堂上也讲了, 如下:

```

1  #include<vector>
2  using namespace std;
3  int main(){
4      /*
5       * 在课堂上将了vector为何物。本质上说它就是一个数组。
6       * 但是数组有int型数组, 如 int a[5]
7       * 也有float型数组, 如float[5]
8       * 但是vector只有一种, 如何满足既要灵活、又要与类型固定。
9       * 下面的 <> 中就是模版。
10     */
11
12     vector<int> iv;
13     vector<float> fv;
14
15     return 0;
16 }

```

**Library (库) :**

库就是库。

## 6.向量 vector

```
1  #include<vector>
2  using namespace std;
3  int main(){
4      vector<int> iv;
5
6      //如下讲解CURD
7      iv.push_back(1); //给这个向量增加了一个
8      int i = 0;
9      for(;i<10;i++){
10         iv.push_back(i+100); // 增加了10个
11     }
12     iv.erase(iv.begin()+2); // 删除了index=2的元素。
13
14     for(i = 0;i<iv.size();i++){
15         cout<<iv[i];          //遍历
16     }
17     return 0;
18 }
```

如何在vector中添加一个自定数据类型。

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
4  struct Person{
5      int id;
6  };
7  int main(){
8      vector<Person> person_vector;
9      Person p;
10     p.id = 10;
11     person_vector.push_back(p);
12     cout<<person_vector[0].id<<endl;
13     return 0;
14 }
```

## 7.文件操作

C++ 的文件操作相对来说很简单。

```

1  #include <fstream>
2  #include <iostream>
3  #include <string>
4  using namespace std;
5  int main() {
6      ifstream file("d:\\123.txt"); // 尝试打开文件
7      if (!file.is_open()) { // 检查文件是否成功打开
8          cerr << "无法打开文件" << std::endl;
9          return -1; // 或其他错误处理代码
10     }
11     string line; // 用于存储每行的内容
12     while (getline(file, line)) { // 逐行读取文件内容
13         cout << line << endl; // 输出到控制台或进行其他处理
14     }
15     file.close(); // 关闭文件流以释放资源
16     return 0; // 程序正常结束
17 }

```

每次读取一行，这样的读取方式仅适用于文本文件，不适合二进制文件。读取二进制我们就不说了。

读取的文件，我们可以用一个 `vector` 保持起来。

```

1  #include <fstream>
2  #include <iostream>
3  #include <string>
4  #include <vector>
5  using namespace std;
6  int main() {
7      ifstream file("d:\\123.txt");
8      if (!file.is_open()) {
9          cerr << "无法打开文件" << std::endl;
10         return -1;
11     }
12     string line;
13     vector<string> lines;
14     while (getline(file, line)) {
15         lines.push_back(line);
16     }
17     file.close();
18     int i = 0;
19     for(; i < lines.size(); i++){
20         cout << lines[i] << endl;
21     }
22     return 0;
23 }

```